PBA: 5. Data Wrangling

Fall 2025

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Roadmap

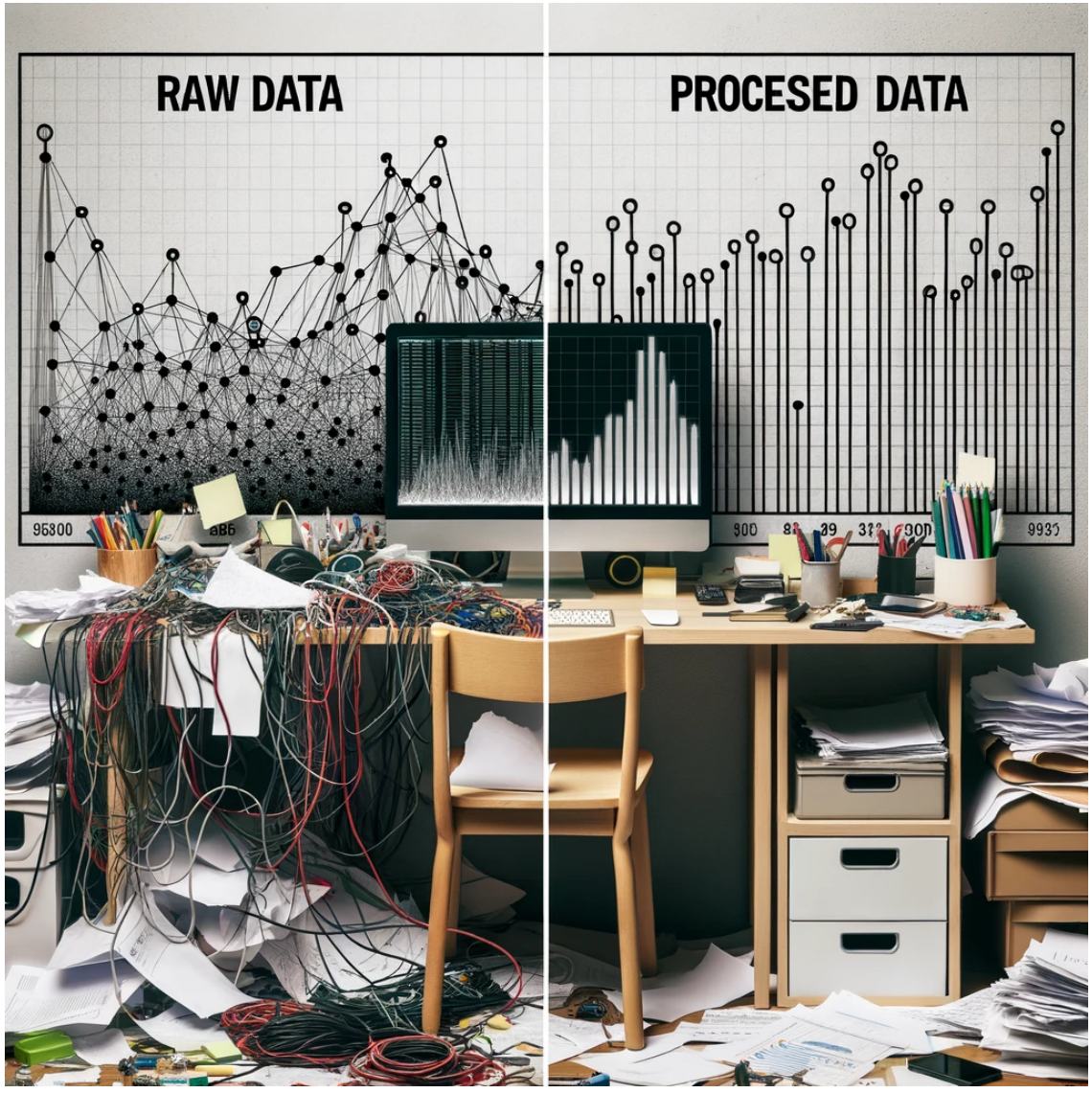
1. Operating on Rows

2. Operating on Columns

3. Operating on Groups

4. Creating Barplots

Data Wrangling $\rightsquigarrow$ Q: Why Do It?



Source: created using a generative AI, DALLE-3!

Prompt I used: I want to visualize how real-world data is very messy. We need data wrangling before we can model it and extract insights. Can you help create a visualization illustrating this?

data.frames vs. tibbles

- The standard R object for datasets is the `data.frame`
  - Each column is a vector of the same length
  - Columns can be different types

- Access columns with `$`: `my_data$my_variable`
- **Example:**

```
> mtcars$mpg

 [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4
[17] 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7 15.0 21.4
```

Problems with Data Frames

```
> mtcars
                     mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Mazda RX4           21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag       21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
Datsun 710          22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive      21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout   18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
Valiant             18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
Duster 360          14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
Merc 240D           24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
Merc 230            22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
Merc 280            19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
Merc 280C           17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
Merc 450SE          16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
Merc 450SL          17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
Merc 450SLC         15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
Fiat 128            32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
Honda Civic         30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
Toyota Corolla      33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
Toyota Corona       21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
Dodge Challenger    15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
AMC Javelin         15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
Camaro Z28          13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
Pontiac Firebird    19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
Fiat X1-9           27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
Porsche 914-2       26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
Lotus Europa        30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
```

tibbles: A Tidyverse Alternative

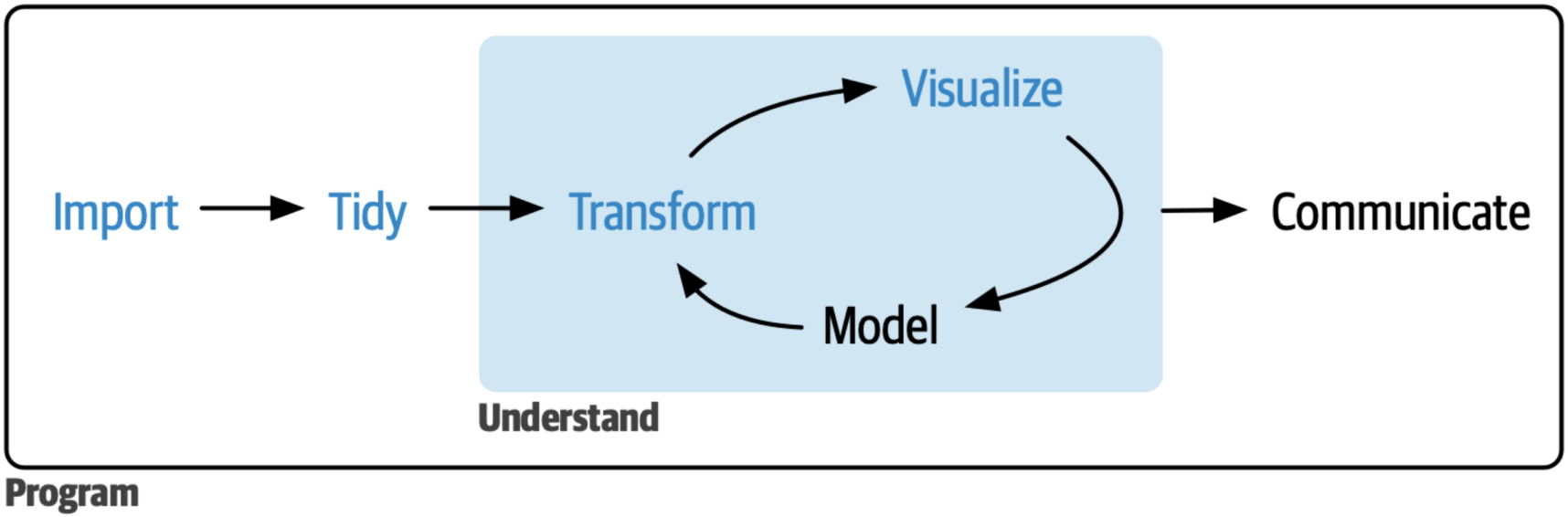
```
> midwest
# A tibble: 437 × 28
     PID county    state  area poptotal popdensity popwhite popblack popamerindian
   <int> <chr>     <chr> <dbl>    <int>      <dbl>    <int>    <int>          <int>
 1   561 ADAMS     IL    0.052    66090      1271.    63917     1702             98
 2   562 ALEXANDER IL    0.014    10626       759      7054     3496             19
 3   563 BOND      IL    0.022    14991       681.    14477      429             35
 4   564 BOONE     IL    0.017    30806      1812.    29344      127             46
 5   565 BROWN     IL    0.018     5836       324.     5264      547             14
 6   566 BUREAU    IL    0.05     35688       714.    35157       50             65
 7   567 CALHOUN   IL    0.017     5322       313.     5298        1              8
 8   568 CARROLL   IL    0.027    16805       622.    16519      111             30
 9   569 CASS      IL    0.024    13437       560.    13384       16              8
10   570 CHAMPAIGN IL    0.058   173025      2983.   146506    16559            331
#  427 more rows
#  19 more variables: popasian <int>, popother <int>, percwhite <dbl>,
#   percblack <dbl>, percamerindan <dbl>, percasian <dbl>, percother <dbl>,
#   popadults <int>, perchsd <dbl>, percollege <dbl>, percprof <dbl>,
#   poppovertyknown <int>, percpovertyknown <dbl>, percbelowpoverty <dbl>,
#   percchildbelowpovert <dbl>, percadultpoverty <dbl>, percelderlypoverty <dbl>,
#   inmetro <int>, category <chr>
#  Use `print(n = ...)` to see more rows
```

rows x columns

column types

abridged output

The Transform-Visualize-Model Cycle



Credit: Hadley Wickham

dplyr: A Package for Data Transformation

All dplyr functions:

1. Take a dataset as their first argument
2. Manipulate the dataset in some way
3. Returns the manipulated dataset

The Pipe

- Nested calls can be hard to read! (have to read inside out):

```
f(g(h(r(x))))
```

- The pipe |> allows us to move output between functions (i.e., |> = “and then”):

```
x |>
  r() |>
  h() |>
  g() |>
  h()
```

- The piped output goes to the first argument of the subsequent function by default.

Local News Data

- How does station ownership affect local news coverage?
- Martin and McCrain (2019) use data on local news at TV stations before and after a large acquisition by a conglomerate.

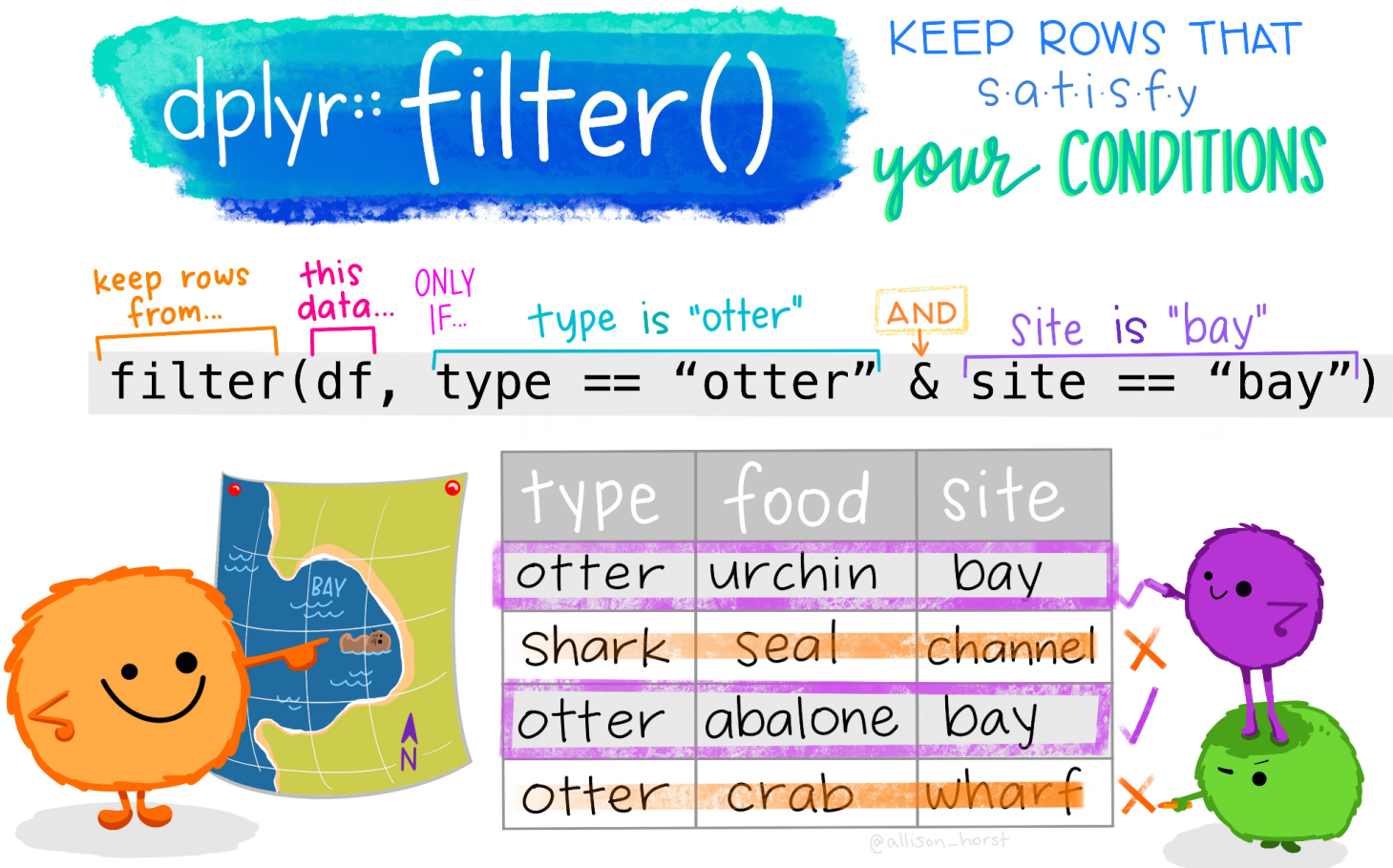
Variable	Description
`callsign`	Callsign of the station (i.e., unique station identifier)
`affiliation`	Network affiliation of the station (e.g., ABC, FOX News)
`date`	Airdate of news
`weekday`	Day of the week of airdate
`ideology`	Measure of news slant (bigger is more conservative)
`national_politics`	Avg proportion of segments on national politics
`local_politics`	Avg proportion of segments on local politics
`sinclair2017`	Station acquired by Sinclair group in Sept 2017
`post`	Date is before/after acquisition (0/1)

```
> news <- as_tibble(read.csv(url("https://bit.ly/3Qst0Ry"))); news

# A tibble: 2,560 × 10
   callsign affiliation date       weekday  ideology national_politics local_politics
   <chr>    <chr>       <chr>      <chr>       <dbl>             <dbl>          <dbl>
 1 KECI     NBC         2017-06-07 Wed      0.0655              0.225          0.148
 2 KPAX     CBS         2017-06-07 Wed      0.0853              0.283          0.123
 3 KRBC     NBC         2017-06-07 Wed      0.0183              0.130          0.189
 4 KTAB     CBS         2017-06-07 Wed      0.0850              0.0901         0.138
 5 KTMF     ABC         2017-06-07 Wed      0.0842              0.152          0.129
 6 KTXS     ABC         2017-06-07 Wed     -0.000488            0.0925         0.0791
 7 KAEF     ABC         2017-06-08 Thu      0.0426              0.213          0.228
 8 KBVU     FOX         2017-06-08 Thu     -0.0860              0.169          0.247
 9 KECI     NBC         2017-06-08 Thu      0.0902              0.276          0.152
10 KPAX     CBS         2017-06-08 Thu      0.0668              0.305          0.124
#  2,550 more rows
#  3 more variables: sinclair2017 <int>, post <int>, month <chr>
#  Use `print(n = ...)` to see more rows
```

1/ Operating on Rows

filter()

`filter()` selects rows that satisfy the argument you pass it:



Credit: Allison Horst

```
> news |>
    filter(weekday == "Tue")

# A tibble: 509 × 10
   callsign affiliation date       weekday ideology national_politics local_politics
   <chr>    <chr>       <date>     <ord>      <dbl>             <dbl>          <dbl>
 1 KAEF     ABC         2017-06-13 Tue      0.0242             0.180          0.234
 2 KBVU     FOX         2017-06-13 Tue      0.00894            0.186          0.245
 3 KBZK     CBS         2017-06-13 Tue      0.129              0.306          0.0763
 4 KCVU     FOX         2017-06-13 Tue      0.114              0.124          0.178
 5 KECI     NBC         2017-06-13 Tue      0.115              0.283          0.0926
 6 KHSL     CBS         2017-06-13 Tue      0.0821             0.274          0.248
 7 KNVN     NBC         2017-06-13 Tue      0.120              0.261          0.253
 8 KPAX     CBS         2017-06-13 Tue      0.0984             0.208          0.171
 9 KRCR     ABC         2017-06-13 Tue      0.0187             0.206          0.232
10 KTMF     ABC         2017-06-13 Tue      0.233              0.101          0.144
#  499 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

Multiple Conditions Means “and”!

```
> news |>
    filter(weekday == "Tue",
           affiliation == "FOX")

# A tibble: 70 × 10
   callsign affiliation date       weekday ideology national_politics local_politics
   <chr>    <chr>       <date>     <ord>      <dbl>             <dbl>          <dbl>
 1 KBVU     FOX         2017-06-13 Tue      0.00894             0.186          0.245
 2 KCVU     FOX         2017-06-13 Tue      0.114               0.124          0.178
 3 WEMT     FOX         2017-06-13 Tue      0.235               0.149          0.155
 4 WYDO     FOX         2017-06-13 Tue      0.0949              0.182          0.180
 5 WEMT     FOX         2017-06-20 Tue      0.268               0.134          0.151
 6 WYDO     FOX         2017-06-20 Tue      0.0590              0.155          0.0943
 7 WEMT     FOX         2017-06-27 Tue      0.0457              0.164          0.171
 8 WYDO     FOX         2017-06-27 Tue     -0.0588              0.147          0.123
 9 WEMT     FOX         2017-07-04 Tue      0.0801              0.0986         0.180
10 WYDO     FOX         2017-07-04 Tue      0.0555              0.0910         0.169
#  60 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

Common Mistake!

```
> news |>
    filter(weekday = "Tue")
```

```
Error in `filter()`:
! We detected a named input.
 This usually means that you've used `=` instead of `==`.
 Did you mean `weekday == "Tue"`?
Run `rlang::last_trace()` to see where the error occurred.
```

Recall: Logicals

- Comparing two values/vectors:
  - >/>=: greater than/greater than or equal to
  - </<=: less than/less than or equal to
  - ==/! =: equal to/not equal to

- Combining multiple logical statements:
  - &: and (conjunction)
  - |: or (disjunction)

Filtering with Multiple Conditions: $cond_a$ and $cond_b$

```
> news |>
    filter(weekday == "Tue" & ideology < 0)

# A tibble: 1,033 × 10
   callsign affiliation date       weekday  ideology national_politics local_politics
   <chr>    <chr>       <date>     <ord>       <dbl>             <dbl>          <dbl>
 1 KTMF     ABC         2017-06-07 Wed      0.0842              0.152          0.129
 2 KTXS     ABC         2017-06-07 Wed     -0.000488            0.0925         0.0791
 3 KAEF     ABC         2017-06-08 Thu      0.0426              0.213          0.228
 4 KBVU     FOX         2017-06-08 Thu     -0.0860              0.169          0.247
 5 KTMF     ABC         2017-06-08 Thu      0.0433              0.179          0.139
 6 KTXS     ABC         2017-06-08 Thu      0.0627              0.158          0.115
 7 WCTI     ABC         2017-06-08 Thu      0.139               0.225          0.0759
 8 KAEF     ABC         2017-06-09 Fri      0.0870              0.153          0.269
 9 KTXS     ABC         2017-06-09 Fri      0.0879              0.0790         0.131
10 WCTI     ABC         2017-06-09 Fri      0.0667              0.182          0.111
#  1,023 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

Filtering with Multiple Conditions: $cond_1$ or $cond_2$

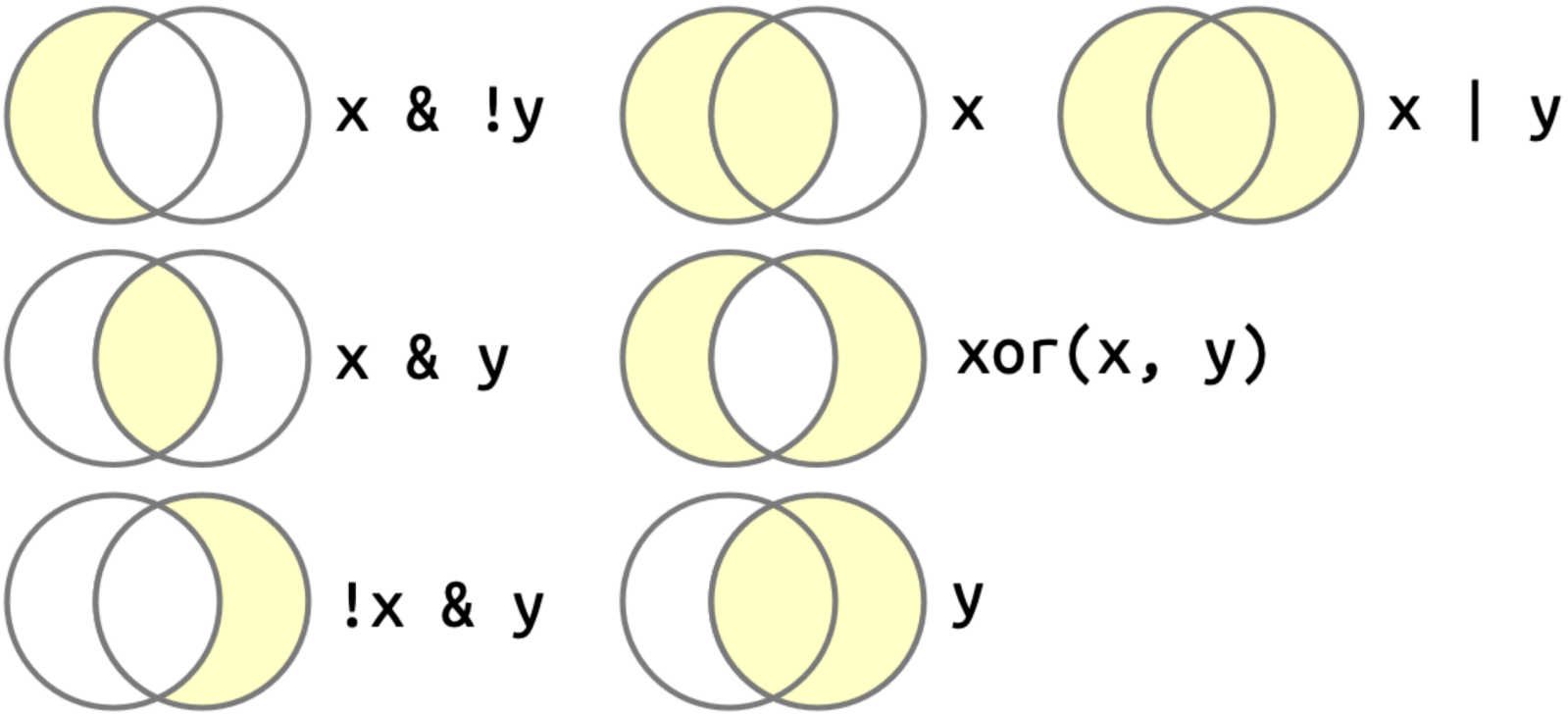
```
> news |>
    filter(affiliation == "FOX" | affiliation == "ABC")

# A tibble: 1,033 × 10
   callsign affiliation date       weekday  ideology national_politics local_politics
   <chr>    <chr>       <date>     <ord>       <dbl>             <dbl>          <dbl>
 1 KTMF     ABC         2017-06-07 Wed      0.0842              0.152          0.129
 2 KTXS     ABC         2017-06-07 Wed     -0.000488            0.0925         0.0791
 3 KAEF     ABC         2017-06-08 Thu      0.0426              0.213          0.228
 4 KBVU     FOX         2017-06-08 Thu     -0.0860              0.169          0.247
 5 KTMF     ABC         2017-06-08 Thu      0.0433              0.179          0.139
 6 KTXS     ABC         2017-06-08 Thu      0.0627              0.158          0.115
 7 WCTI     ABC         2017-06-08 Thu      0.139               0.225          0.0759
 8 KAEF     ABC         2017-06-09 Fri      0.0870              0.153          0.269
 9 KTXS     ABC         2017-06-09 Fri      0.0879              0.0790         0.131
10 WCTI     ABC         2017-06-09 Fri      0.0667              0.182          0.111
#  1,023 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

Combining %in%

- When you want to check if a value is equal to any one of several options, you can simplify it using %in%.
  - a %in% B, where B$\in\{1,2,3\}$ is a == 1 | a == 2 | a == 3
  - Equivalent to 'combining' | and ==

```
> news |>
    filter(weekday %in% c("mon", "Fri"))

# A tibble: 500 × 10
   callsign affiliation date       weekday ideology national_politics local_politics
   <chr>    <chr>       <date>     <ord>      <dbl>             <dbl>          <dbl>
 1 KAEF     ABC         2017-06-09 Fri       0.0870            0.153          0.269
 2 KECI     NBC         2017-06-09 Fri       0.115             0.216          0.108
 3 KPAX     CBS         2017-06-09 Fri       0.0882            0.315          0.128
 4 KRBC     NBC         2017-06-09 Fri       0.0929            0.152          0.147
 5 KTAB     CBS         2017-06-09 Fri       0.0588            0.0711         0.176
 6 KTXS     ABC         2017-06-09 Fri       0.0879            0.0790         0.131
 7 WCTI     ABC         2017-06-09 Fri       0.0667            0.182          0.111
 8 WITN     NBC         2017-06-09 Fri      -0.0683            0.146          0.185
 9 WNCT     CBS         2017-06-09 Fri      -0.0181            0.126          0.198
10 WYDO     FOX         2017-06-09 Fri      -0.0330            0.0802         0.194
#  490 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

Other Complicated Logicals



arrange()

`arrange()` *will reorder the rows based on the values of the columns. With multiple arguments, sort by first argument, then second, then third...*

Arrange by `callsign` then `date`

```
> news |>
    arrange(callsign, date)

# A tibble: 2,560 × 10
   callsign affiliation date       weekday ideology national_politics local_politics
   <chr>    <chr>       <date>     <ord>      <dbl>             <dbl>          <dbl>
 1 KAEF     ABC         2017-06-08 Thu       0.0426             0.213          0.228
 2 KAEF     ABC         2017-06-09 Fri       0.0870             0.153          0.269
 3 KAEF     ABC         2017-06-12 Mon       0.0135             0.149          0.188
 4 KAEF     ABC         2017-06-13 Tue       0.0242             0.180          0.234
 5 KAEF     ABC         2017-06-14 Wed       0.123              0.182          0.0968
 6 KAEF     ABC         2017-06-15 Thu       0.0778             0.114          0.203
 7 KAEF     ABC         2017-06-19 Mon       0.778              0.0823         0.179
 8 KAEF     ABC         2017-06-20 Tue       0.115              0.131          0.163
 9 KAEF     ABC         2017-06-21 Wed      -0.315              0.130          0.192
10 KAEF     ABC         2017-06-22 Thu       0.0873             0.104          0.189
#  2,550 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

1/ Which Station-Date were the Most Liberal?

```
> news |>
    arrange(ideology)

# A tibble: 2,560 × 10
   callsign affiliation date       weekday ideology national_politics local_politics
   <chr>    <chr>       <date>     <ord>      <dbl>             <dbl>          <dbl>
 1 KRBC     NBC         2017-10-19 Thu       -0.674            0.0731          0.161
 2 WJHL     CBS         2017-12-08 Fri       -0.673            0.0364          0.206
 3 KRBC     NBC         2017-10-18 Wed       -0.586            0.0470          0.135
 4 KCVU     FOX         2017-06-22 Thu       -0.414            0.158           0.172
 5 KRBC     NBC         2017-12-11 Mon       -0.365            0.0674          0.163
 6 KAEF     ABC         2017-06-21 Wed       -0.315            0.130           0.192
 7 KTMF     ABC         2017-12-01 Fri       -0.303            0.179           0.150
 8 KWYB     ABC         2017-12-01 Fri       -0.303            0.160           0.161
 9 KTVM     NBC         2017-09-01 Fri       -0.302            0.0507          0.106
10 KNVN     NBC         2017-12-08 Fri       -0.299            0.121           0.211
#  2,550 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

- Political affiliation: Liberal ⟷ Conservative
  - Liberal: low `ideology` score
  - Conservative: high `ideology` score

2/ Which Station-Date were the Most Conservative?

```
> news |>
    arrange(desc(ideology))

# A tibble: 2,560 × 10
   callsign affiliation date       weekday ideology national_politics local_politics
   <chr>    <chr>       <date>     <ord>      <dbl>             <dbl>          <dbl>
 1 KAEF     ABC         2017-06-19 Mon        0.778            0.0823          0.179
 2 WYDO     FOX         2017-07-19 Wed        0.580             0.126          0.121
 3 KRCR     ABC         2017-10-03 Tue        0.566             0.123          0.192
 4 KAEF     ABC         2017-10-18 Wed        0.496            0.0892          0.217
 5 KBVU     FOX         2017-11-16 Thu        0.491             0.159          0.184
 6 KTMF     ABC         2017-11-06 Mon        0.455             0.138          0.154
 7 KAEF     ABC         2017-06-29 Thu        0.447             0.126          0.220
 8 KPAX     CBS         2017-11-23 Thu        0.437             0.125          0.128
 9 KTAB     CBS         2017-11-16 Thu        0.427            0.0631          0.104
10 KCVU     FOX         2017-07-06 Thu        0.406             0.154          0.148
#  2,550 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

- Political affiliation: Liberal ⟷ Conservative
  - Liberal: low `ideology` score
  - Conservative: high `ideology` score

slice()

- `slice()` can give you a specific set of rows:

```
> news |>
    slice(1,3)

# A tibble: 2 × 10
  callsign affiliation date       weekday ideology national_politics local_politics
  <chr>    <chr>       <date>     <ord>      <dbl>             <dbl>          <dbl>
1 KECI     NBC         2017-06-07 Wed       0.0655             0.225          0.148
2 KRBC     NBC         2017-06-07 Wed       0.0183             0.130          0.189
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
```

- You can ask for a range of rows with `start:stop` syntax:

```
> news |>
    slice(1:3)

# A tibble: 3 × 10
  callsign affiliation date       weekday ideology national_politics local_politics
  <chr>    <chr>       <date>     <ord>      <dbl>             <dbl>          <dbl>
1 KECI     NBC         2017-06-07 Wed       0.0655             0.225          0.148
2 KPAX     CBS         2017-06-07 Wed       0.0853             0.283          0.123
3 KRBC     NBC         2017-06-07 Wed       0.0183             0.130          0.189
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
```

slice_min() & slice_max()

- `slice_min(var, n = k)` will return the bottom *k* observations on column `var`:
  - Likewise, `slice_max(var, n = k)` will return the top *k* obs.

```
> news |>
    slice_min(ideology, n = 5)

# A tibble: 5 × 10
  callsign affiliation date       weekday ideology national_politics local_politics
  <chr>    <chr>       <date>     <ord>      <dbl>             <dbl>          <dbl>
1 KRBC     NBC         2017-10-19 Thu       -0.674            0.0731          0.161
2 WJHL     CBS         2017-12-08 Fri       -0.673            0.0364          0.206
3 KRBC     NBC         2017-10-18 Wed       -0.586            0.0470          0.135
4 KCVU     FOX         2017-06-22 Thu       -0.414            0.158           0.172
5 KRBC     NBC         2017-12-11 Mon       -0.365            0.0674          0.163
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
```

2/ Operating on Columns

select()

- select() selects columns via their names.
- Select based on names (left) or based on a range of variables (right):

```
> news |>
    select(callsign, date, ideology)

# A tibble: 2,560 × 3
   callsign date        ideology
   <chr>    <date>         <dbl>
 1 KECI     2017-06-07  0.0655
 2 KPAX     2017-06-07  0.0853
 3 KRBC     2017-06-07  0.0183
 4 KTAB     2017-06-07  0.0850
 5 KTMF     2017-06-07  0.0842
 6 KTXS     2017-06-07 -0.000488
 7 KAEF     2017-06-08  0.0426
 8 KBVU     2017-06-08 -0.0860
 9 KECI     2017-06-08  0.0902
10 KPAX     2017-06-08  0.0668
#  2,550 more rows
#  Use `print(n = ...)` to see more rows
```

```
> news |>
    select(callsign:date)

# A tibble: 2,560 × 3
   callsign affiliation date
   <chr>    <chr>       <date>
 1 KECI     NBC         2017-06-07
 2 KPAX     CBS         2017-06-07
 3 KRBC     NBC         2017-06-07
 4 KTAB     CBS         2017-06-07
 5 KTMF     ABC         2017-06-07
 6 KTXS     ABC         2017-06-07
 7 KAEF     ABC         2017-06-08
 8 KBVU     FOX         2017-06-08
 9 KECI     NBC         2017-06-08
10 KPAX     CBS         2017-06-08
#  2,550 more rows
#  Use `print(n = ...)` to see more rows
```

1/ More Use Cases of `select()`

- Selecting all that are not (i.e., !) in a given range:

```
> news |>
    select(!callsign:date)

# A tibble: 2,560 × 7
   weekday  ideology national_politics local_politics sinclair2017  post month
   <ord>       <dbl>             <dbl>          <dbl>        <dbl> <dbl> <ord>
 1 Wed      0.0655              0.225          0.148            1     0 Jun
 2 Wed      0.0853              0.283          0.123            0     0 Jun
 3 Wed      0.0183              0.130          0.189            0     0 Jun
 4 Wed      0.0850              0.0901         0.138            0     0 Jun
 5 Wed      0.0842              0.152          0.129            0     0 Jun
 6 Wed     -0.000488            0.0925         0.0791           1     0 Jun
 7 Thu      0.0426              0.213          0.228            1     0 Jun
 8 Thu     -0.0860              0.169          0.247            1     0 Jun
 9 Thu      0.0902              0.276          0.152            1     0 Jun
10 Thu      0.0668              0.305          0.124            0     0 Jun
#  2,550 more rows
#  Use `print(n = ...)` to see more rows
```

2/ More Use Cases of `select()`

- Selecting all numeric columns:
  - `dplyr::where()` takes a condition and returns a list of variables that satisfy it.

```
> news |>
    select(where(is.numeric))

# A tibble: 2,560 × 5
    ideology national_politics local_politics sinclair2017  post
       <dbl>             <dbl>          <dbl>        <int> <int>
 1  0.0655              0.225          0.148            1     0
 2  0.0853              0.283          0.123            0     0
 3  0.0183              0.130          0.189            0     0
 4  0.0850              0.0901         0.138            0     0
 5  0.0842              0.152          0.129            0     0
 6 -0.000488          0.0925         0.0791           1     0
 7  0.0426              0.213          0.228            1     0
 8 -0.0860              0.169          0.247            1     0
 9  0.0902              0.276          0.152            1     0
10  0.0668              0.305          0.124            0     0
#  2,550 more rows
#  Use `print(n = ...)` to see more rows
```

3/ More Use Cases of `select()`

- Combining multiple selections:
  - `dplyr::ends_with()` selects columns whose names end with a specified string.
  - Similarly, `dplyr::starts_with()` (or `dplyr::contains()`) points to columns with names that begin with (or contains) a specified string,

```
> news |>
    select(callsign:weekday, ends_with("politics"))

# A tibble: 2,560 × 6
   callsign affiliation date       weekday national_politics local_politics
   <chr>    <chr>       <date>     <ord>              <dbl>          <dbl>
 1 KECI     NBC         2017-06-07 Wed               0.225          0.148
 2 KPAX     CBS         2017-06-07 Wed               0.283          0.123
 3 KRBC     NBC         2017-06-07 Wed               0.130          0.189
 4 KTAB     CBS         2017-06-07 Wed               0.0901         0.138
 5 KTMF     ABC         2017-06-07 Wed               0.152          0.129
 6 KTXS     ABC         2017-06-07 Wed               0.0925         0.0791
 7 KAEF     ABC         2017-06-08 Thu               0.213          0.228
 8 KBVU     FOX         2017-06-08 Thu               0.169          0.247
 9 KECI     NBC         2017-06-08 Thu               0.276          0.152
10 KPAX     CBS         2017-06-08 Thu               0.305          0.124
#  2,550 more rows
#  Use `print(n = ...)` to see more rows
```

rename()

- `rename(new_name = old_name)` renames the `old_name` variable to `new_name`.

```
> news |>
    rename(call_sign = callsign)

# A tibble: 2,560 × 10
   call_sign affiliation date       weekday  ideology national_politics local_politics
   <chr>     <chr>       <date>     <ord>       <dbl>             <dbl>          <dbl>
 1 KECI      NBC         2017-06-07 Wed      0.0655              0.225          0.148
 2 KPAX      CBS         2017-06-07 Wed      0.0853              0.283          0.123
 3 KRBC      NBC         2017-06-07 Wed      0.0183              0.130          0.189
 4 KTAB      CBS         2017-06-07 Wed      0.0850              0.0901         0.138
 5 KTMF      ABC         2017-06-07 Wed      0.0842              0.152          0.129
 6 KTXS      ABC         2017-06-07 Wed     -0.000488            0.0925         0.0791
 7 KAEF      ABC         2017-06-08 Thu      0.0426              0.213          0.228
 8 KBVU      FOX         2017-06-08 Thu     -0.0860              0.169          0.247
 9 KECI      NBC         2017-06-08 Thu      0.0902              0.276          0.152
10 KPAX      CBS         2017-06-08 Thu      0.0668              0.305          0.124
#  2,550 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

mutate()

- `mutate(new_var = fun(old_vars))` adds new columns that are functions of existing columns.

```
> news |>
    mutate(
      national_local_diff = national_politics - local_politics,
      national_politics_perc = national_politics * 100
    ) |>
    select(callsign, date, national_politics, local_politics,
           national_local_diff, national_politics_perc)

# A tibble: 2,560 × 6
   callsign date       national_politics local_politics national_local_diff
   <chr>    <date>                 <dbl>          <dbl>               <dbl>
 1 KECI     2017-06-07             0.225          0.148              0.0761
 2 KPAX     2017-06-07             0.283          0.123              0.160
 3 KRBC     2017-06-07             0.130          0.189             -0.0589
 4 KTAB     2017-06-07             0.0901         0.138             -0.0476
 5 KTMF     2017-06-07             0.152          0.129              0.0229
 6 KTXS     2017-06-07             0.0925         0.0791             0.0134
 7 KAEF     2017-06-08             0.213          0.228             -0.0151
 8 KBVU     2017-06-08             0.169          0.247             -0.0781
 9 KECI     2017-06-08             0.276          0.152              0.124
10 KPAX     2017-06-08             0.305          0.124              0.180
#  2,550 more rows
#  1 more variable: national_politics_perc <dbl>
#  Use `print(n = ...)` to see more rows
```

if_else()

- `if_else(test_condition, yes, no)` allows us to create a vector that depends on a logical.
  - New vector gets yes expression when test_condition is TRUE, no otherwise.

```
> news |>
    mutate(Ownership = if_else(sinclair2017 == 1,
                               "Acquired by Sinclair",
                               "Not Acquired")) |>
    select(callsign, affiliation, date, Ownership)

# A tibble: 2,560 × 4
   callsign affiliation date       Ownership
   <chr>    <chr>       <date>     <chr>
 1 KECI     NBC         2017-06-07 Acquired by Sinclair
 2 KPAX     CBS         2017-06-07 Not Acquired
 3 KRBC     NBC         2017-06-07 Not Acquired
 4 KTAB     CBS         2017-06-07 Not Acquired
 5 KTMF     ABC         2017-06-07 Not Acquired
 6 KTXS     ABC         2017-06-07 Acquired by Sinclair
 7 KAEF     ABC         2017-06-08 Acquired by Sinclair
 8 KBVU     FOX         2017-06-08 Acquired by Sinclair
 9 KECI     NBC         2017-06-08 Acquired by Sinclair
10 KPAX     CBS         2017-06-08 Not Acquired
#  2,550 more rows
#  Use `print(n = ...)` to see more rows
```

3/ Operating on Groups

group_by()

- group_by(var) divides the data into groups based on the var variable:
  - Doesn't change the data yet, but subsequent operations will be done by groups specified in var.

```
> news |>
    group_by(month)

# A tibble: 2,560 × 10
# Groups:   month [7]
   callsign affiliation date       weekday  ideology national_politics local_politics
   <chr>    <chr>       <date>     <ord>       <dbl>             <dbl>          <dbl>
 1 KECI     NBC         2017-06-07 Wed      0.0655              0.225          0.148
 2 KPAX     CBS         2017-06-07 Wed      0.0853              0.283          0.123
 3 KRBC     NBC         2017-06-07 Wed      0.0183              0.130          0.189
 4 KTAB     CBS         2017-06-07 Wed      0.0850              0.0901         0.138
 5 KTMF     ABC         2017-06-07 Wed      0.0842              0.152          0.129
 6 KTXS     ABC         2017-06-07 Wed     -0.000488            0.0925         0.0791
 7 KAEF     ABC         2017-06-08 Thu      0.0426              0.213          0.228
 8 KBVU     FOX         2017-06-08 Thu     -0.0860              0.169          0.247
 9 KECI     NBC         2017-06-08 Thu      0.0902              0.276          0.152
10 KPAX     CBS         2017-06-08 Thu      0.0668              0.305          0.124
#  2,550 more rows
#  3 more variables: sinclair2017 <dbl>, post <dbl>, month <ord>
#  Use `print(n = ...)` to see more rows
```

summarize()

- `summarize(sum_var = fun(curr_var))` calculates summaries of variables:

```
> news |>
    group_by(month) |>
    summarize(
      political_leaning_mean = mean(ideology, na.rm = TRUE)
    )

# A tibble: 7 × 2
  month political_leaning_mean
  <ord>                  <dbl>
1 Jun                   0.0786
2 Jul                   0.103
3 Aug                   0.105
4 Sep                   0.0751
5 Oct                   0.0862
6 Nov                   0.0972
7 Dec                   0.0774
```

Summaries by Ownership and Pre/Post

```
> news |>
    group_by(sinclair2017, post) |>
    summarize(
      slant_mean = mean(ideology, na.rm = TRUE),
      national_mean = mean(national_politics, na.rm = TRUE)
    )

# A tibble: 4 × 4
# Groups:   sinclair2017 [2]
  sinclair2017  post slant_mean national_mean
         <dbl> <dbl>      <dbl>         <dbl>
1            0     0     0.100          0.134
2            0     1     0.0768         0.126
3            1     0     0.0936         0.137
4            1     1     0.0938         0.155
```

Summaries Across Types of Variables

- across() will apply a summary function across many variables:

```
> news |>
    group_by(sinclair2017, post) |>
    summarize(
      across(where(is.numeric), mean, na.rm = TRUE)
    )

# A tibble: 4 × 5
# Groups:   sinclair2017 [2]
  sinclair2017  post ideology national_politics local_politics
         <dbl> <dbl>    <dbl>             <dbl>          <dbl>
1            0     0   0.100              0.134          0.168
2            0     1   0.0768             0.126          0.167
3            1     0   0.0936             0.137          0.157
4            1     1   0.0938             0.155          0.139
```

kable() to produce nice tables

```
> news |>
    group_by(month) |>
    summarize(
      political_leaning_mean = mean(ideology, na.rm = TRUE) ) |>
    knitr::kable(col.names = c("Month", "Avg. Ideology"))   # we can also specify column names
```

Month	Avg. Ideology
Jun	0.0785518
Jul	0.1032917
Aug	0.1049908
Sep	0.0751067
Oct	0.0861639
Nov	0.0971796
Dec	0.0773873

Producing a Table of Counts for a Categorical Var.

- First begin by group_by(), then `summarize(n = n())` to count the observations, or...

```
> news |>
    group_by(affiliation) |>
    summarize(n = n())
# A tibble: 4 × 2
  affiliation     n
  <chr>       <int>
1 ABC           687
2 CBS           758
3 FOX           346
4 NBC           769
```

- Just use `count()`!

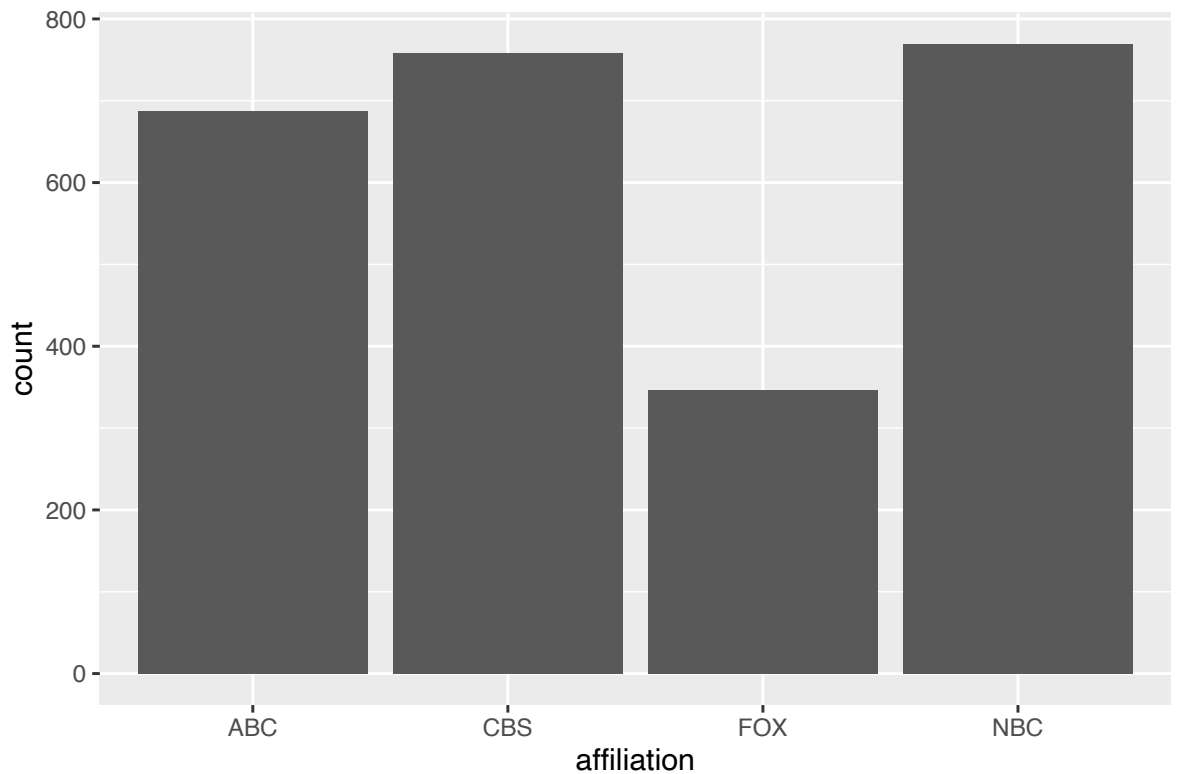
```
> news |>
    count(affiliation)

# A tibble: 4 × 2
  affiliation     n
  <chr>       <int>
1 ABC           687
2 CBS           758
3 FOX           346
4 NBC           769
```

4/ Creating Barplots

Combining Our Skills

- Let's combine our tools to produce a bar plot with `geom_bar()`.
- Recall: By default, bar plots take a single variable and show the number of observations in each category.

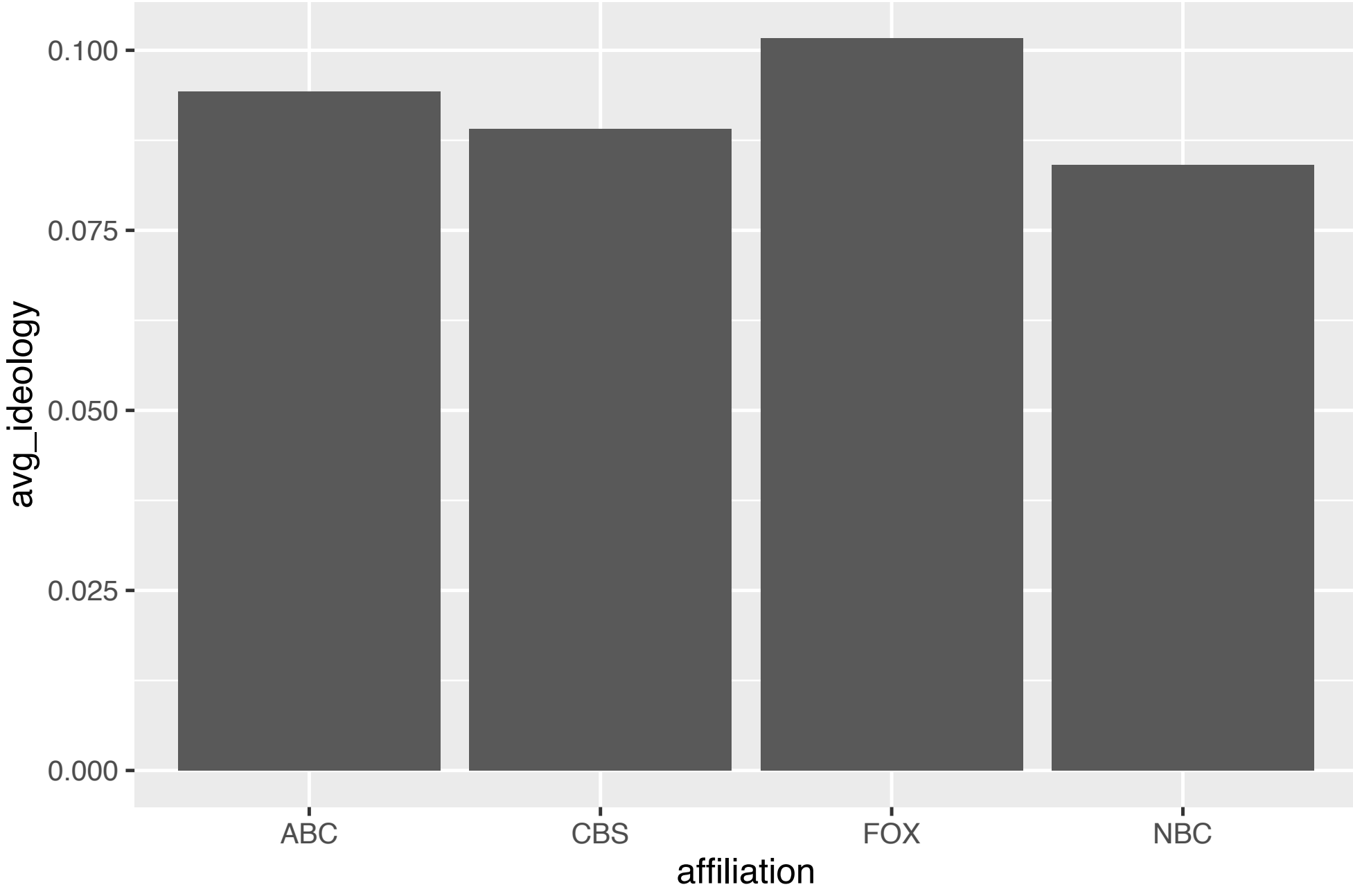
```
> ggplot(news, aes(x = affiliation)) +
    geom_bar()
```



Barplots of Non-Counts

- Barplots can represent a lot beyond counts, including variables in our dataset or group summaries.
  - When the height of the bar is another variable in our data and not just a count, we set the x and y aesthetics and use `geom_col()` instead of `geom_bar()`.
- Let's create a group summary then plot it:

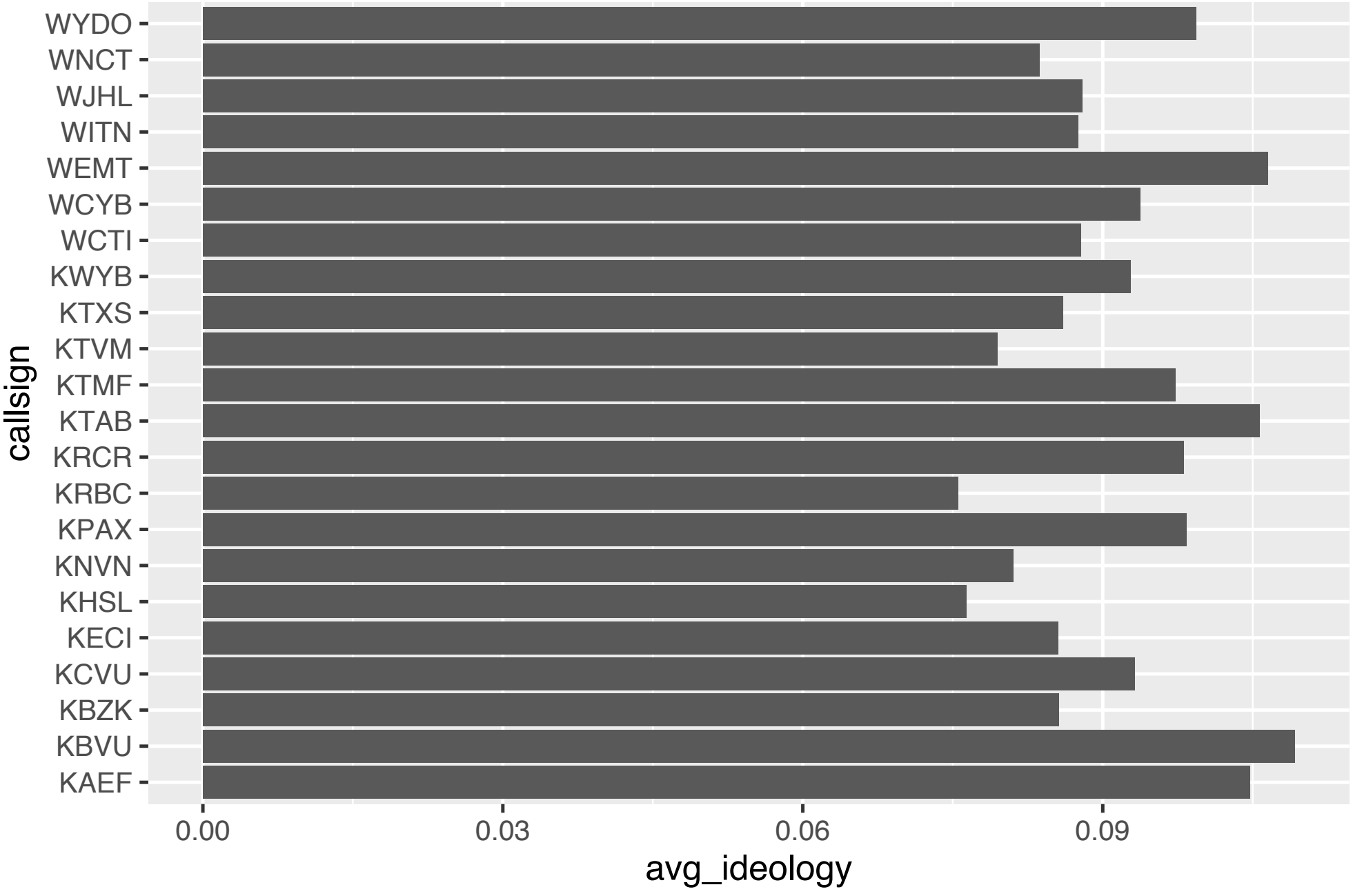
```
> aff_ideology_means <- news |>
    group_by(affiliation) |>
    summarize(avg_ideology = mean(ideology, na.rm = TRUE))
> aff_ideology_means

# A tibble: 4 × 2
  affiliation avg_ideology
  <chr>              <dbl>
1 ABC               0.0943
2 CBS               0.0891
3 FOX               0.102
4 NBC               0.0841

> ggplot(aff_ideology_means, aes(x = affiliation, y = avg_ideology)) +
    geom_col()
```


0.100
0.075
0.050
0.025
0.000
avg_ideology
ABC
CBS
FOX
NBC
affiliation

Slightly More Complicated Example

- Let's create a barplot that shows the top 20 stations in terms of their political leaning (i.e., ideology).
- First, let's get the data then plot:

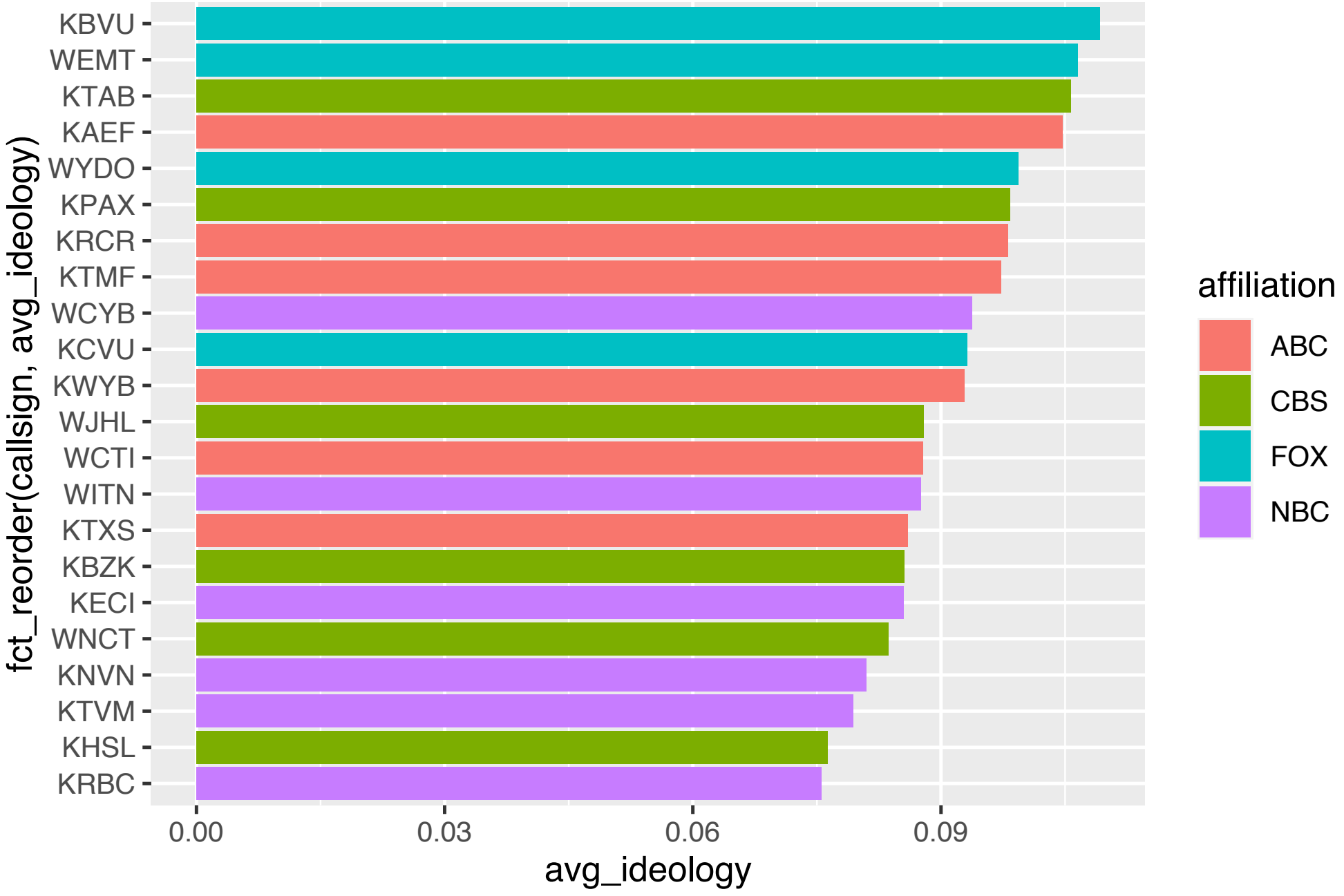
```
> station_ideology <- news |>
    group_by(callsign, affiliation) |>
    summarize(avg_ideology = mean(ideology, na.rm = TRUE)) |>
    slice_max(avg_ideology, n = 20)

> ggplot(station_ideology, aes(x = avg_ideology, y = callsign)) +
    geom_col()
```


WYDO
WNCT
WJHL
WITN
WEMT
WCYB
WCTI
KWYB
KTXS
KTVM
KTMF
KTAB
KRCR
KRBC
KPAX
KNVN
KHSL
KECI
KCVU
KBZK
KBVU
KAEF
callsign
0.00
0.03
0.06
0.09
avg_ideology

How Do We Reorder the Stations?

- We'd like to order the stations by ideology.
- `fct_reorder(group, order_var)` function (loaded with `tidyverse`) will reorder the groups by the order bar (low to high).
    - Make sure to put it in the mapping (i.e., `aes()`).

```
> ggplot(data = station_ideology,
         mapping = aes(x = avg_ideology,
                       y = fct_reorder(callsign, avg_ideology))) +
    geom_col(aes(fill = affiliation))
```


KBVU
WEMT
KTAB
KAEF
WYDO
KPAX
KRCR
KTMF
WCYB
KCVU
KWYB
WJHL
WCTI
WITN
KTXS
KBZK
KECI
WNCT
KNVN
KTVM
KHSL
KRBC
fct_reorder(callsign, avg_ideology)
0.00
0.03
0.06
0.09
avg_ideology
affiliation
ABC
CBS
FOX
NBC

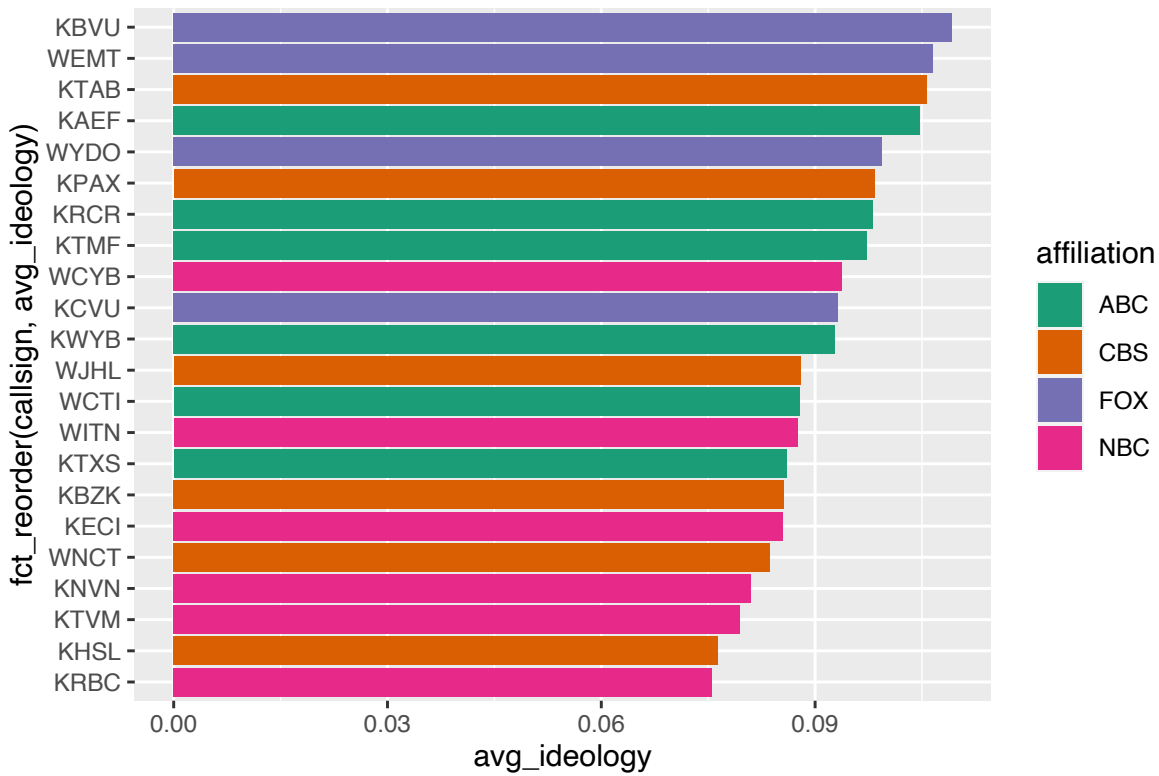
Enjoy your Weekends! :)

Contact Information:
jaewon.yoo@iss.nthu.edu.tw
https://j1yoo.github.io/

Setting the Color Palette

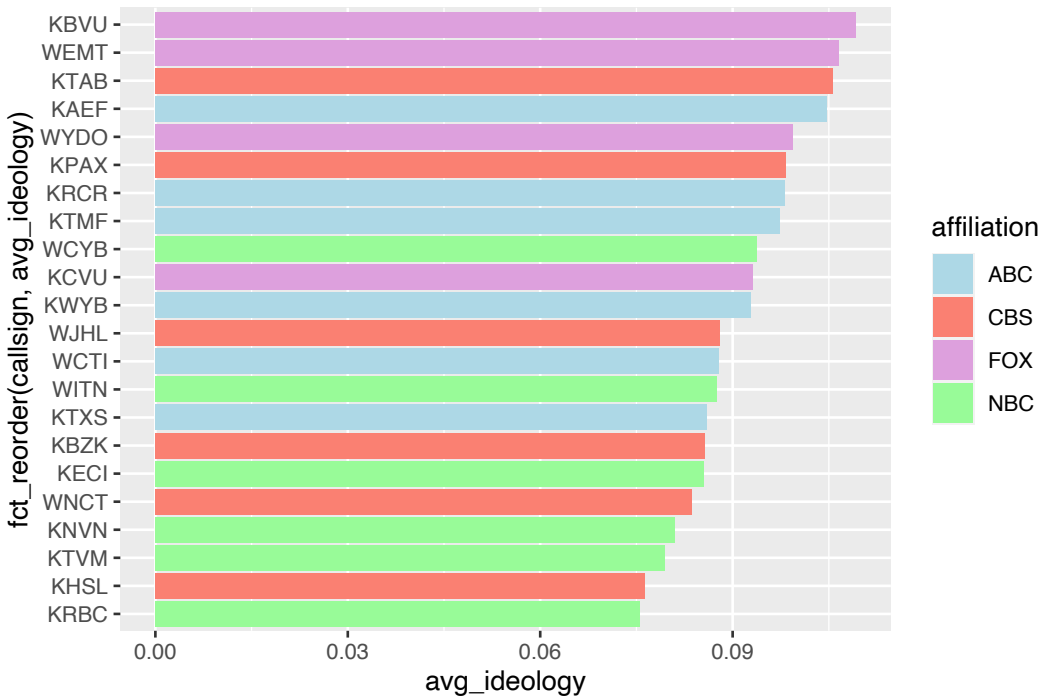
- We can use color palettes from a 'project'/built-in package called ColorBrewer:

```
> ggplot(station_ideology,
         mapping = aes(x = avg_ideology,
                       y = fct_reorder(callsign, avg_ideology))) +
    geom_col(mapping = aes(fill = affiliation)) +
    scale_fill_brewer(palette = "Dark2")
```



Manually Setting the Color Palette

```
> ggplot(station_ideology,
         mapping = aes(x = avg_ideology,
                       y = fct_reorder(callsign, avg_ideology))) +
    geom_col(mapping = aes(fill = affiliation)) +
    scale_fill_manual(values = c(ABC = "lightblue",
                                 CBS = "salmon",
                                 FOX = "plum",
                                 NBC = "palegreen"))
```



Fun with Colors

```
> library(wesanderson)   # install the package first!
> ggplot(station_ideology,
         mapping = aes(x = avg_ideology,
                       y = fct_reorder(callsign, avg_ideology))) +
    geom_col(mapping = aes(fill = affiliation)) +
    scale_fill_manual(values = wes_palette("Moonrise3"))
```